ЛЕКЦИЯ · АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

Коллекции Python и сложность операций

Почему `x in set` в тысячи раз быстрее, чем `x in list`



БЛОК 1 · ВСТУПЛЕНИЕ

Один вопрос, разница в тысячи раз

```
banned = list(range(100_000))
banned_set = set(range(100_000))

%timeit 99_999 in banned     # вариант A
%timeit 99_999 in banned_set # вариант B
```

list · вариант A

сотни микросекунд

set · вариант B

десятки наносекунд

БЛОК 1 · ВСТУПЛЕНИЕ

Интерфейс и устройство

ЧТО УМЕЕТ

Абстрактный тип данных

Набор операций и их смысл: вставить, найти, удалить, достать минимум.

КАК УСТРОЕНО

Структура данных

Способ хранения в памяти. Именно он задаёт стоимость операций.

В Python они склеены: выбирая list, вы выбираете динамический массив, выбирая set — хеш-таблицу.

БЛОК 1 · ВСТУПЛЕНИЕ

Как оценивать операции

`O(1)` < `O(log n)` < `O(n)` < `O(n log n)` < `O(n²)`

Худший случай

Гарантия при любых данных

`deque.popleft()`

Средний случай

Типичное поведение, как у хеш-таблиц

`d[key]`

Амортизированная

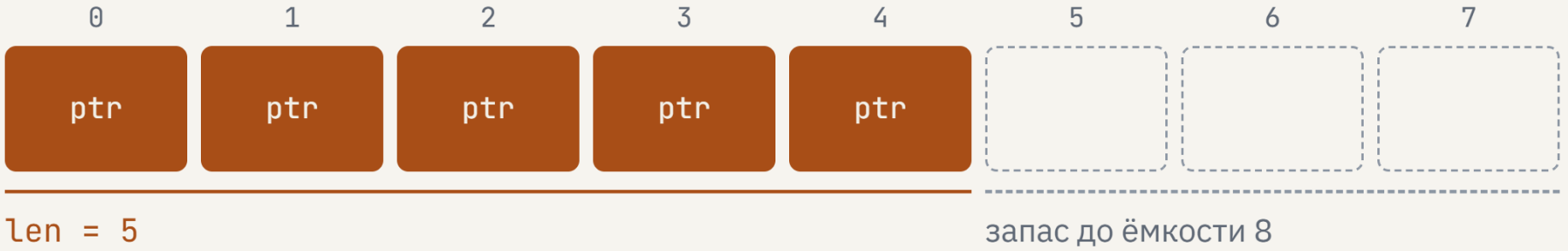
Средняя цена в длинной серии

`list.append(x)`

Измеряем: **%timeit** — время, **sys.getsizeof** — память самого контейнера, без объектов внутри.

БЛОК 2 · LIST

list — массив указателей с запасом



Доступ по индексу — O(1)

Адрес a[i] = начало + i × 8 байт. Считается, а не ищется.

Хранятся указатели

Поэтому в одном списке живут числа и строки, а копия списка поверхностная.

append: редко дорого, в среднем O(1)

```
import sys
a, last = [], None
for i in range(64):
    size = sys.getsizeof(a)
    if size ≠ last:
        print(f"len={len(a):3}  bytes={size}")
        last = size
    a.append(i)
```

Размер растёт **скачками**, а не на каждом шаге.

Место кончилось → новый блок примерно на 12,5% больше → копирование указателей за O(n).

Рост геометрический, копирования редки: **O(1) амортизированно**.

Вопрос: что, если расти на фиксированные +10 ячеек?

БЛОК 2 · LIST

Дешёвые и дорогие операции

Операция	Время	Почему
a[i], a[i] = x, len(a)	O(1)	адрес вычисляется
append(x), pop()	O(1) аморт.	работа с концом
insert(0, x), pop(0), del a[i]	O(n)	сдвиг всего хвоста
x in a, index, remove, count	O(n)	линейный просмотр
a[i:j], extend(k элементов)	O(k)	срез — это копия
sort()	O(n log n)	Timsort, устойчивая

Timsort на почти отсортированных данных приближается к O(n).

БЛОК 2 · LIST

Конец списка против начала

```
a = list(range(1_000_000))
%timeit a.append(0); a.pop()     # десятки нс
%timeit a.insert(0, 0); a.pop(0) # сотни мкс
```

Работай с концом списка. Нужно начало — бери deque.

Ловушка

`[[0] * 3] * 3`

Три ссылки на один и тот же внутренний список.

БЛОК 3 · TUPLE И STR

tuple — неизменяемый массив указателей

Без запаса — компактнее

```
getsizeof((1, 2, 3)) <
getsizeof([1, 2, 3])
```

Сложность как у list

Индекс и len — O(1), in и index — O(n). Изменяющих операций нет.

Хешируемый

Если элементы хешируемы, годится в ключи:

```
grid[(x, y)] = cell
```

Запись, а не коллекция

list — однородные элементы переменной длины, tuple — фиксированные поля. Именованные поля: namedtuple.

str: склейка в цикле — скрытый квадрат

```
# O(n²): каждый += создаёт новую строку
def concat(n):
    s = ""
    for i in range(n):
        s += str(i)
    return s
```

```
# O(n): собрать части и склеить один раз
def joined(n):
    return "".join(str(i) for i in range(n))
```

Строка — неизменяемый массив символов шириной 1, 2 или 4 байта (PEP 393), поэтому **s[i] — O(1)**.

sub in s — поиск подстроки, на практике близко к линейному.

Оговорка. CPython иногда расширяет строку на месте, и замер может не показать разницу. Это оптимизация реализации, а не гарантия языка.

Поиск вычислением адреса, а не перебором

1
key
Ключ, который ищем

→

2
hash(key)
Целое число

→

3
ячейка
По сути хеш по модулю размера

→

4
hash и ==
Проверка совпадения

Коллизия

Ячейка занята другим ключом → следующая по псевдослучайной последовательности (открытая адресация).

Рост

Заполнена примерно на 2/3 → увеличивается и перестраивается. Как у list: O(1) амортизированно.

Ключ не должен меняться

```
>>> {[1, 2]: 'x'}
TypeError: unhashable type: 'list'
>>> {(1, 2): 'x'}              # работает
```

Замены

tuple вместо list, frozenset вместо set.

Свои классы

Переопределили __eq__ — нужен согласованный __hash__.

Порядок

dict помнит порядок вставки с 3.7, set — нет.

O(1) в среднем, O(n) в худшем

```
class Bad:
    def __init__(self, v): self.v = v
    def __hash__(self): return 42
    def __eq__(self, o): return self.v == o.v

%timeit {Bad(i) for i in range(2000)}
%timeit {i for i in range(2000)}
```

Все ключи попадают в одну ячейку — каждый поиск перебирает цепочку проб.

Построение множества становится квадратичным.

Из-за атак на коллизии хеши строк рандомизированы при каждом запуске: PYTHONHASHSEED.

Операции над множествами и их цена

Операция	Время
a | b	O(len(a) + len(b))
a & b	O(min(len(a), len(b)))
a - b	O(len(a))

Пересечение перебирает меньшее множество и ищет в большем.

```
# дубликаты прочь, порядок сохранён
list(dict.fromkeys(items))
```

ЦЕНА СКОРОСТИ

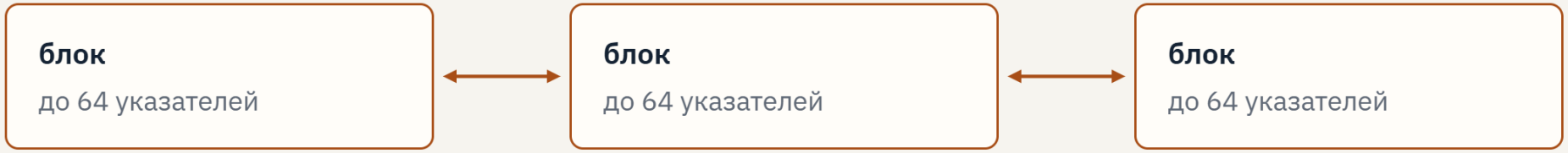
Множество в несколько раз больше списка

```
getsizeof(list(range(1000)))
getsizeof(set(range(1000)))
```

Свободные ячейки и хранимые хеши. Классический компромисс «время против памяти».

БЛОК 5 · COLLECTIONS

deque — быстро с обоих концов



O(1) в худшем случае

`append, appendleft, pop, popleft`

Расплата

d[i] в середине — O(n), срезов нет.

Где нужен

Очередь для BFS, deque(maxlen=N) — последние N событий.

Надстройки над dict — сложность та же

Counter

```
c = Counter(words)
c.most_common(10)
```

Подсчёт — O(n). Топ-k — O(n log k) через кучу.

defaultdict

```
g = defaultdict(list)
g[key].append(x)
```

Группировка без проверок «if key not in d».

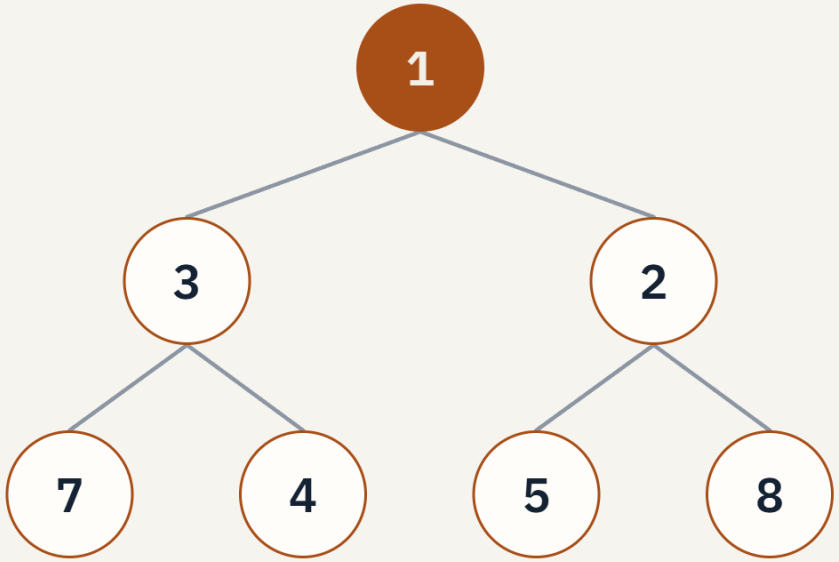
OrderedDict

```
od.move_to_end(k)
od.popitem(last=False)
```

После 3.7 нужен редко, но move_to_end за O(1) удобен для LRU-кэша.

БЛОК 5 · HEAPQ

heapq — минимум всегда наверху



В памяти это обычный list: [1, 3, 2, 7, 4, 5, 8]

Операция	Время
heappush, heappop	O(log n)
heap[0]	O(1)
heapify	O(n)

Сортированный список даёт вставку за O(n), куча — за O(log n).

Макс-куча — отрицательные ключи. Задачи: топ-k, Дейкстра, планировщик.

БЛОК 5 · BISECT

bisect — двоичный поиск в отсортированном

```
from bisect import bisect

def grade(score):
    bounds = [60, 70, 80, 90]
    return "FDCBA"[bisect(bounds, score)]

grade(85)  # 'B'
```

bisect_left / bisect_right — O(log n).

insort — O(log n) на поиск места, но O(n) на сдвиг при вставке.

Сложность операции = сумма её шагов.

В стандартной библиотеке нет сбалансированного дерева. Упорядоченный словарь с вставкой за O(log n) — sortedcontainers. Компактные массивы чисел — array или NumPy.

БЛОК 6 · СВОДНАЯ ТАБЛИЦА

Сводка: доступ, поиск, вставка

Операция	list	tuple	deque	dict	set
Доступ по индексу / ключу	O(1)	O(1)	O(1) края, O(n) середина	O(1) ср.	—
x in c	O(n)	O(n)	O(n)	O(1) ср.	O(1) ср.
Добавить в конец / вставить	O(1) ам.	—	O(1)	O(1) ам.	O(1) ам.
Добавить в начало	O(n)	—	O(1)	—	—
Удалить с конца	O(1)	—	O(1)	—	—
Удалить с начала	O(n)	—	O(1)	—	—
Вставить / удалить в середине	O(n)	—	O(n)	—	—

CPython · «ам.» — амортизированно · «ср.» — в среднем, худший случай хеш-таблиц O(n) · «—» — операции нет

БЛОК 6 · СВОДНАЯ ТАБЛИЦА

Сводка: удаление, перебор, память

Операция	list	tuple	deque	dict	set
Удалить по значению / ключу	O(n)	—	O(n)	O(1) ср.	O(1) ср.
len(c)	O(1)	O(1)	O(1)	O(1)	O(1)
Копирование, перебор	O(n)	O(n)	O(n)	O(n)	O(n)
Срез длины k	O(k)	O(k)	—	—	—
Сортировка	O(n log n)	—	—	—	—
Память на элемент	малая + запас	минимальная	малая	высокая	высокая

CPython · «ср.» — в среднем, худший случай хеш-таблиц O(n) · «—» — операции нет · проверить себя: wiki.python.org/moin/TimeComplexity

БЛОК 6 · СВОДНАЯ ТАБЛИЦА

Сводка: инструменты

Инструмент	Операция	Время
heapq	heappush, heappop	O(log n)
heapq	heapify	O(n)
heapq	минимум h[0]	O(1)
bisect	bisect_left, bisect_right	O(log n)
bisect	insort	O(n)
set	a | b	O(len(a) + len(b))
set	a & b	O(min(len(a), len(b)))
str	''.join(parts)	O(суммарная длина)
Counter	most_common(k)	O(n log k)

БЛОК 7 · ПРАКТИКУМ

Выбери коллекцию под задачу

Задача	Нужные операции	Ответ
Видел ли краулер этот URL	добавить, проверить вхождение	set
Очередь задач FIFO	в конец, из начала	deque
Очередь с приоритетами	добавить, взять самую срочную	heapq
Частота слов, топ-10	увеличить счётчик, топ-k	Counter
Координаты точки как ключ	неизменяемая запись, хеш	tuple
Последние 100 действий	добавить, вытеснить старое	deque(maxlen)
Рейтинг: редко меняется, часто поиск позиции	поиск в отсортированном	list + bisect

Найди квадрат: где здесь O(n²)?

```
# 1. Уникальные элементы
unique = []
for x in items:
    if x not in unique:
        unique.append(x)
```

```
# 2. Обработка очереди
while queue:
    task = queue.pop(0)
    process(task)
```

```
# 3. Пересечение двух списков
common = [x for x in a if x in b]
```

```
# 4. Сборка отчёта
report = ""
for row in rows:
    report += format_row(row)
```

БЛОК 7 · ПРАКТИКУМ

Операция O(n) внутри цикла по n элементам — это O(n²)

1 · уникальные

```
seen = set()
или dict.fromkeys
```

2 · очередь

```
deque
+ popleft()
```

3 · пересечение

```
b_set = set(b)
до цикла
```

4 · отчёт

```
''.join(
  parts)
```

Проверяйте, что стоит каждая строка в теле цикла.

ИТОГИ

Три мысли, которые стоит унести

1 Сначала набор операций и их частота, потом коллекция.

2 Сложность выводится из устройства: массив, хеш-таблица, список блоков, куча.

3 Скорость оплачивается памятью и ограничениями: хешируемость, нет индекса, нет порядка.

ИТОГИ

Вопросы для самопроверки

1. Почему list.append — O(1), если иногда он копирует весь массив?
2. Почему список нельзя сделать ключом словаря, а кортеж — можно?
3. В каких условиях поиск в dict деградирует до O(n)?
4. Чем deque лучше list и в чём хуже?
5. Почему bisect.insort — O(n), хотя поиск двоичный?

ДОМАШНЕЕ ЗАДАНИЕ

Три задачи до следующей лекции



Замер

x in list и x in set для n = 10^3… 10^6, график в логарифмической шкале.



LRU-кэш

На OrderedDict, get и put за O(1). Объяснить, почему сложность именно такая.



Охота на квадрат

Найти скрытую $O(n^2)$ в своём коде и исправить, с замером до и после.

Источники: wiki.python.org/moin/TimeComplexity · docs.python.org: collections, heapq, bisect